

Manual Técnico e Instalación

BioMove App v4.0

Guía para desarrolladores e instalación del sistema

Versión:	2.0 — Final
Fecha:	13 de junio de 2026
Autor:	Bryan Xavier Calderón Novillo
Institución:	ESPOCH — Ingeniería en Sistemas
Audiencia:	Desarrolladores, administradores del sistema
Stack:	Flutter + FastAPI + SQLite + Celery + Redis + MediaPipe

1. Arquitectura del sistema

BioMove v4.0 implementa arquitectura cliente-servidor de tres capas: (1) Flutter Android (MVVM+Provider), (2) Backend FastAPI (Python 3.10) + Celery 5.4 + Redis, (3) SQLite/PostgreSQL (SQLAlchemy async). El procesamiento de video es asíncrono mediante Celery con Redis como broker. Los videos se almacenan localmente en el servidor (FastAPI StaticFiles). Firebase Auth gestiona exclusivamente la autenticación JWT.

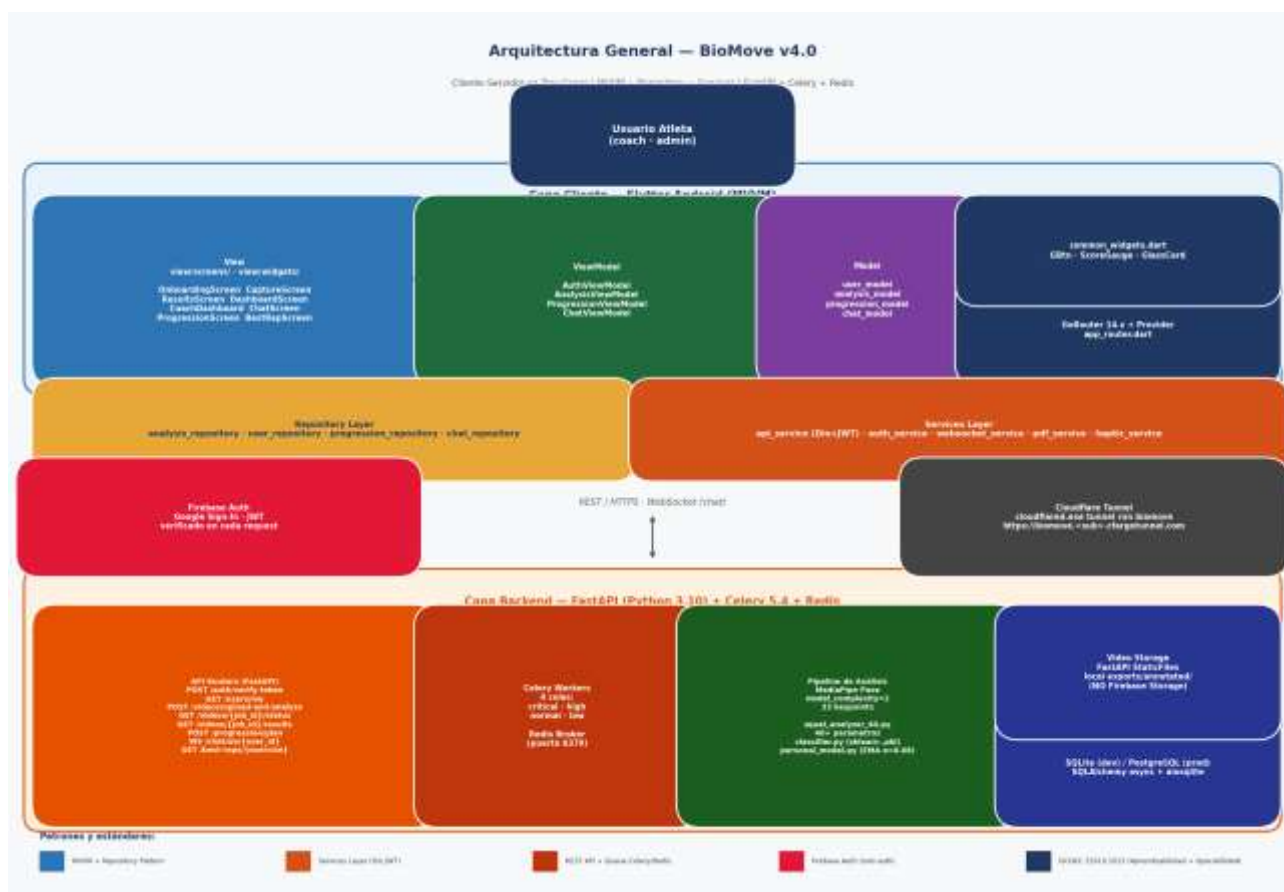


Figura 1. Arquitectura general de BioMove v4.0 — No usar Firebase Storage para videos.

2. Requisitos del sistema

2.1 Servidor backend

Componente	Requisito
Sistema operativo	Windows 10/11 (64-bit) o Linux Ubuntu 22.04+
CPU	4 núcleos mínimo (8 recomendado para Celery + MediaPipe)
RAM	8 GB mínimo (16 GB recomendado)
Python	3.10.x (recomendado 3.10.14)
Docker Desktop	Para Redis (puerto 6379)
Almacenamiento	5 GB libres para videos y base de datos
GPU (opcional)	CUDA 11.x para aceleración de MediaPipe

2.2 Dispositivo cliente (Android)

Android API 29+, 2GB RAM, cámara 720p+, conexión WiFi o datos móviles.

3. Instalación paso a paso

3.1 Clonar el repositorio

Comando
<pre>git clone https://github.com/xaviercalderon/BioMove.git cd BioMove</pre>

3.2 Instalación del backend (Python + FastAPI)

Comandos — Backend (PowerShell o CMD en Windows)
<pre># 1. Crear entorno virtual cd backend python -m venv venv venv\Scripts\activate # Windows # source venv/bin/activate # Linux/Mac # 2. Instalar dependencias pip install -r requirements.txt # 3. Dependencias principales: # fastapi uvicorn sqlalchemy aiosqlite celery redis # mediapipe opencv-python scikit-learn numpy pydantic # firebase-admin python-multipart</pre>

3.3 Configurar variables de entorno (.env)

Crear el archivo backend/.env con las siguientes variables (NUNCA subir a Git):

Archivo: backend/.env
<pre># Base de datos DATABASE_URL=sqlite+aiosqlite:///./biomove.db # En producción: postgresql+asyncpg://user:password@host:5432/biomove # Redis (Celery broker) REDIS_URL=redis://localhost:6379/0 # Firebase (NO incluir credenciales reales aquí) # Colocar firebase-admin-sdk.json en backend/ FIREBASE_CREDENTIALS_PATH=./firebase-admin-sdk.json FIREBASE_STORAGE_BUCKET=biomove-c5ee5.firebaseiostorage.app # Rutas de almacenamiento local UPLOAD_DIR=./uploads EXPORT_DIR=./exports/annotated</pre>

3.4 Configurar Firebase Admin SDK

- Accede a la consola de Firebase (console.firebase.google.com).
- Proyecto "biomove-c5ee5" → Configuración del proyecto → Cuentas de servicio.
- Generar nueva clave privada → descarga el JSON → guárdalo como backend/firebase-admin-sdk.json.

⚠ **IMPORTANTE:** `firebase-admin-sdk.json` está en `.gitignore`. **NUNCA** subir a repositorio público.

3.5 Iniciar Redis con Docker

Comando — Docker Desktop
<pre># Iniciar Redis en Docker (Windows) docker run -d -p 6379:6379 --name redis-biomove redis:alpine # Verificar que Redis está activo: docker ps grep redis</pre>

3.6 Arrancar el sistema completo (start_all.bat)

BioMove incluye `start_all.bat` para iniciar todos los servicios con un solo comando:

backend/start_all.bat — Abre 4 ventanas de terminal
<pre># Ventana 1: FastAPI (API REST) uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload # Ventana 2: Celery Worker (procesamiento de video) python -m celery -A app.workers.tasks worker --loglevel=info --pool=solo # IMPORTANTE: pool=solo en Windows (prefork causa PermissionError WinError 5) # Ventana 3: Flower (monitoreo Celery) python -m celery -A app.workers.tasks flower --port=5555 # Ventana 4: Cloudflare Tunnel (acceso externo opcional) cloudflared.exe tunnel run biomove</pre>

Accesos disponibles: API → `http://localhost:8000` | Docs → `http://localhost:8000/docs` | Flower → `http://localhost:5555`

4. Instalación de la app Flutter

4.1 Requisitos

- Flutter SDK 3.x (stable channel). Instalar desde <https://flutter.dev>
- Android Studio con Android SDK (API 33+ recomendado).
- Un emulador Android (API 29+) o dispositivo físico conectado por USB.

4.2 Configurar URL del backend

Editar frontend/lib/core/utils/app_constants.dart:

app_constants.dart — Configurar baseUrl según entorno
<pre>class AppConstants { // Emulador Android (host = 10.0.2.2 = localhost del PC) static const String baseUrl = 'http://10.0.2.2:8000'; // Dispositivo físico (misma red WiFi): // static const String baseUrl = 'http://192.168.X.X:8000'; // Producción con Cloudflare Tunnel: // static const String baseUrl = 'https://biomove.xxxxx.cfargotunnel.com'; }</pre>

4.3 Ejecutar la app

Comandos Flutter (terminal en carpeta frontend/)
<pre>flutter pub get # Instalar dependencias flutter devices # Ver emuladores/dispositivos disponibles flutter run -d emulator-5554 # Ejecutar en emulador específico flutter build apk --release # Build APK para distribución</pre>

5. Estructura de directorios

Ruta	Contenido
frontend/lib/view/screens/	Pantallas Flutter (DashboardScreen, ResultsScreen, etc.)
frontend/lib/viewmodel/	ViewModels Provider (AnalysisViewModel, AuthViewModel, etc.)
frontend/lib/repository/	Capa de repositorios (abstracción de datos)
frontend/lib/services/	Services: ApiService (Dio), AuthService, WebSocketService
backend/app/routers/	Endpoints FastAPI (videos_router, coach_router, chat_router)
backend/app/analysis/	Pipeline: pipeline.py, squat_analyzer_40.py, video_annotator.py
backend/app/ai/	IA: classifier.py (sklearn), personal_model.py (EMA)
backend/app/models.py	12 modelos SQLAlchemy (User, VideoJob, TrainingSession, etc.)
backend/app/workers/tasks.py	Tareas Celery (análisis async, update_best_rep)
backend/models/squat_clf_v*.pkl	Modelo scikit-learn serializado (5 clases, 43 features)
backend/uploads/{user_id}/	Videos originales subidos por los usuarios

Ruta	Contenido
backend/exports/annotated/	Videos anotados generados por video_annotator.py
backend/firebase-admin-sdk.json	Credenciales Firebase Admin (NO subir a Git)
backend/.env	Variables de entorno (DATABASE_URL, REDIS_URL, etc.)